960121

Microservice Thinking

Session 09

Objective

- Since you have already implemented a **Controller-Route-Service** pattern, you have already achieved "Internal Modularization" (also known as a Clean Monolith).
- Instead of refactoring (separation of logic), the goal of **Session 9** is **Deconstruction**.
- Your current "Modular Monolith" app. is just one step away from becoming a **Microservice Architecture**.

Modularization & Scaling

To think like a systems architect, you need to understand how to **decouple logic**:

- **Monolith vs. Microservices:** Visualizing a single large block of code vs. a network of small, specialized "workers."
- **Service Boundaries:** Deciding where one service ends and another begins (e.g., separating `productLogic.js` from `paymentLogic.js`).
- **Horizontal vs. Vertical Scaling:** Understanding that we can scale a specific service (like Search) during a "Black Friday" sale without scaling the whole application.

1. Modularization: The "Silo" Principle

Modularization is the practice of dividing a program into separate sub-programs. For an e-commerce app, this means the **Authentication Logic** should not "know" how the **Inventory Database** works.

- **The "Clean Desk" Analogy**
  - **Monolith:** Everything is on one desk. If you spill coffee, the computer, the papers, and the phone are all ruined.
  - **Modular:** Each task has its own desk. A spill on the "Marketing" desk doesn't stop the "Accounting" desk from working.

2. Horizontal vs. Vertical Scaling

When the "Business" grows, the "System" must grow.

- **Vertical Scaling (Scaling UP):** Buying a bigger "engine." You add more RAM or a faster CPU to your existing server.
  - ***Limit**:* Eventually, you hit a ceiling; there is only so much power one machine can have.
- **Horizontal Scaling (Scaling OUT):** Adding more "engines." You run five small servers instead of one big one.
  - ***Benefit**:* This is how Amazon or Netflix works. If one server fails, the other four stay up. **Modular code is required for this** because you can't easily copy a "spaghetti" monolith across multiple servers.

3. Separation of Concerns (SoC)

In Session 5, you were taught to organize your folder structure to reflect business boundaries. This is the "**Microservice Mindset**" applied to a single project.

Recommended Folder Structure:

- `services/` — Internal business logic (e.g., `taxCalculator.js`).
- `routes/` — The Gatekeepers (e.g., `productRoutes.js`).
- `controllers/` — The "Brains" that connect routes to services.
- `models/` — The "Data Schema" (The Source of Truth).

Microservices Architecture

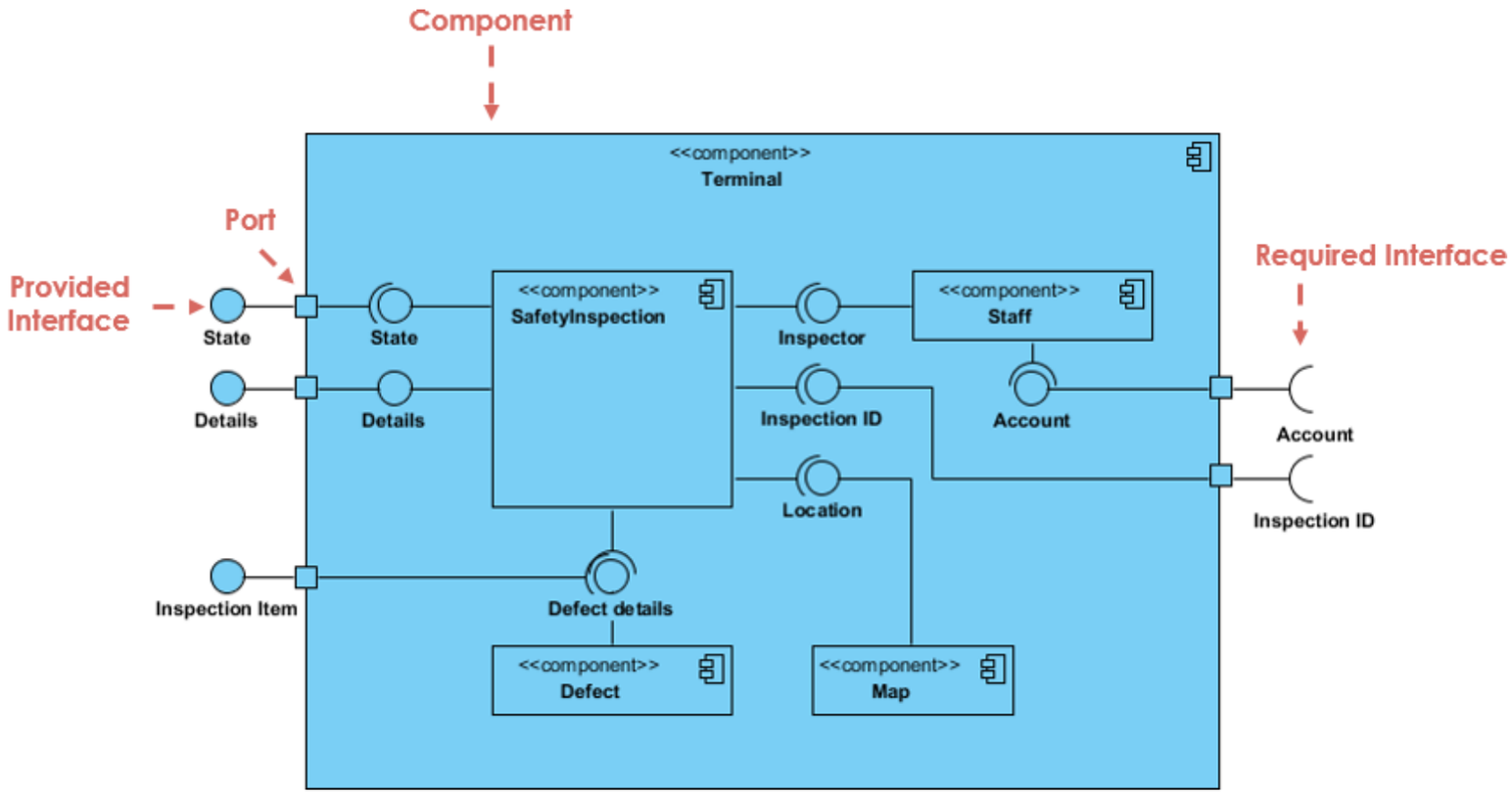
- **Microservices architecture** is a design approach that structures an application as a collection of small, autonomous, and loosely coupled services, usually organized around specific business capabilities.
- Each service runs in its own process and communicates via lightweight mechanisms, often HTTP/REST APIs, allowing for independent deployment and scaling.

Microservices Architecture

- **Independent Deployment & Scalability**: Services are independently deployable, enabling fast, continuous delivery and targeted scaling of specific components rather than the whole app.
- **Specialized and Small**: Focused on a single business function (e.g., payment, catalog), which simplifies understanding and modification.
- **Technological Flexibility**: Different services can use different programming languages, databases, and technology stacks.
- **Resilience**: The failure of one service does not necessarily bring down the entire system.

From Modules to Services

- **Logical vs. Physical Boundaries:** Understanding that while your code is in different files (logical), it still runs on one server (physical).
- **The Shared Fate Problem:** Realizing that in their current project, a memory leak in the `Service` layer of the "Payment" module will still crash the "Product" module because they share the same CPU/RAM.
- **API-First Communication:** Thinking about how modules would talk if they weren't in the same folder—using HTTP or Message Queues instead of `require()`.



Component Diagram.

- A component diagram breaks down the actual system under development into various high levels of functionality. Each component is responsible for one clear aim within the entire system and only interacts with other essential elements on a need-to-know basis.

https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-component-diagram/

Basic Concepts of Component Diagram

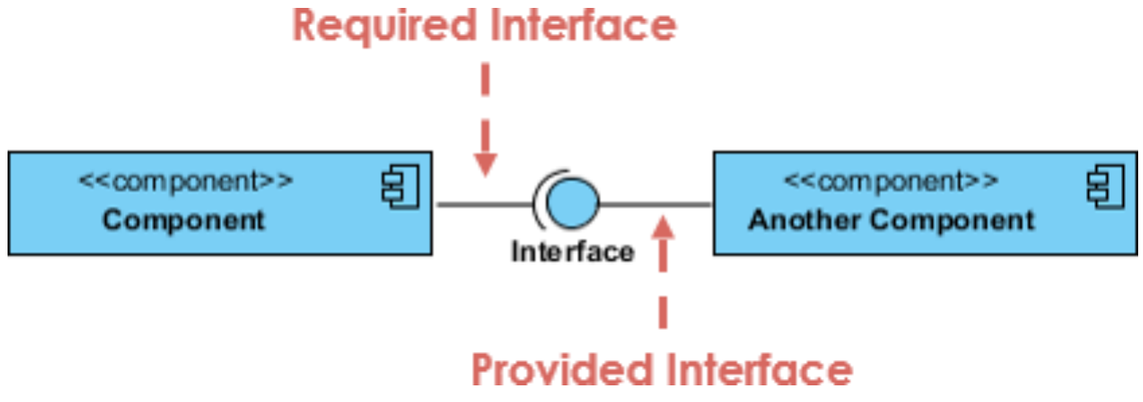
A high-level, abstracted view of a component in UML 2 can be modeled as:

- A rectangle with the component's name
- A rectangle with the component icon
- A rectangle with the stereotype text and/or icon

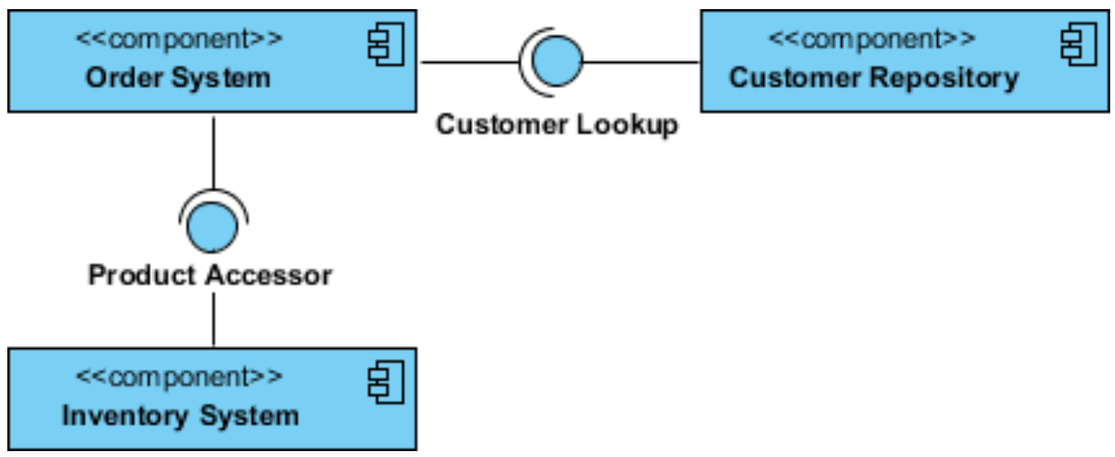
https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-component-diagram/

Interface

- **Provided interface** symbols with a complete circle at their end represent an interface that the component provides.
- **Required Interface** symbols with only a half circle at their end (a.k.a. sockets) represent an interface that the component requires.



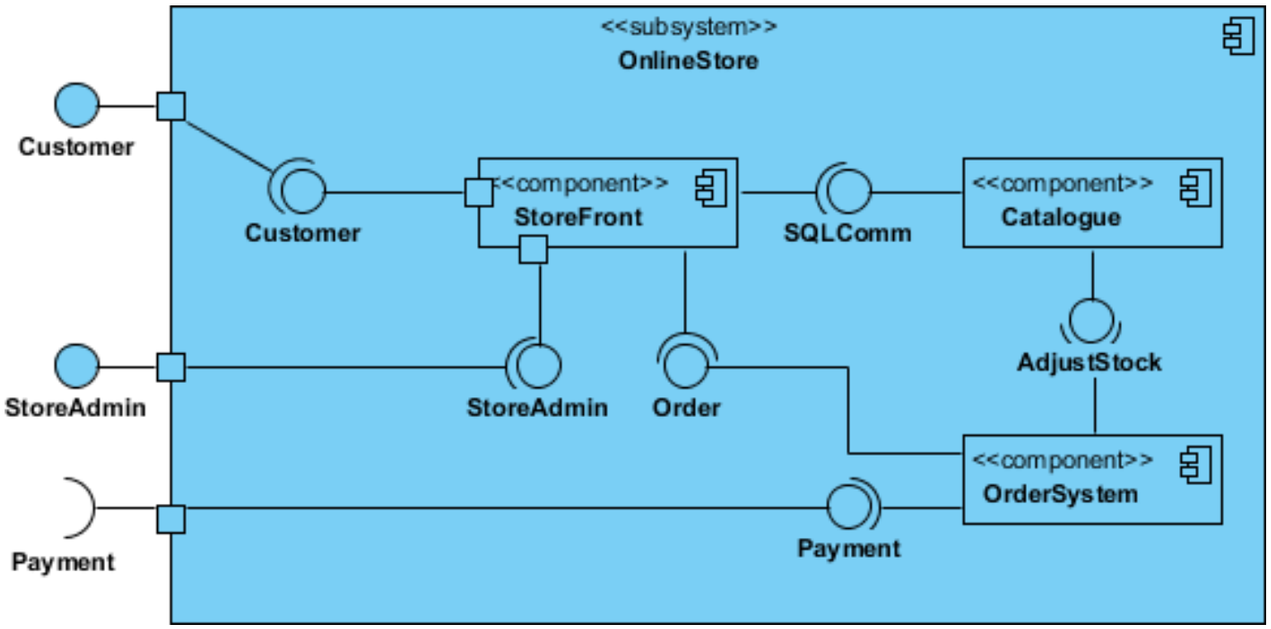
Order System



https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-component-diagram/

Subsystem

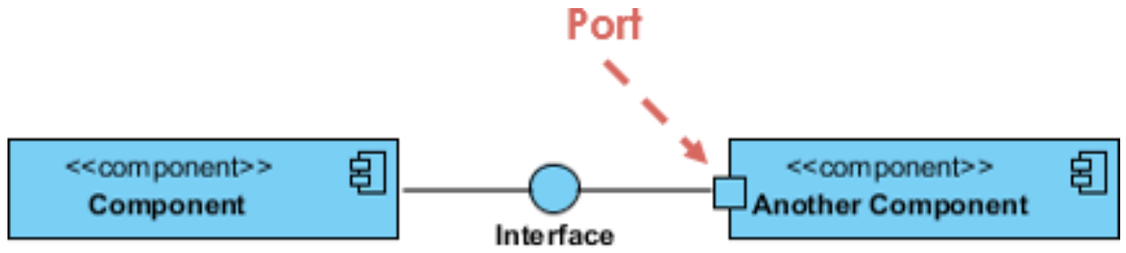
- The subsystem classifier is a specialized version of a component classifier.
- Because of this, the subsystem notation element inherits all the same rules as the component notation element.
- The only difference is that a subsystem notation element has the keyword of subsystem instead of component.



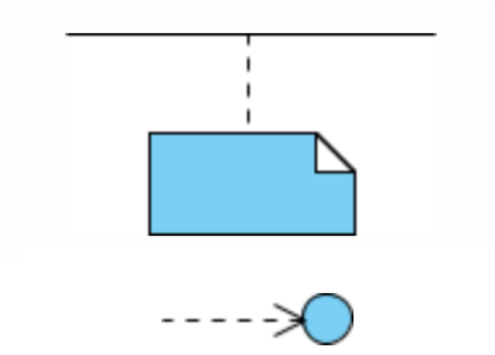
https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-component-diagram/

Port & Relationships

- **Ports** is often used to help expose required and provided interfaces of a component.



- **Relationships**
- Association:
- Composition:
- Aggregation:
- Constraint:
- Dependency:
- Links:



https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-component-diagram/

Referance

- **What is Component Diagram?** (https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-component-diagram/)

Class Activity: Refactoring to Micro-Surgery

1. **The Scenario:**
   - **The Challenge**: Your e-commerce app is a huge success! But the "Product Catalog" team wants to use **Python** for AI recommendations, while the "Checkout" team wants to stay with **Node.js**.
   - **Goal**: We are going to identify how to "cut" your current project into three independent businesses: **Identity**, **Catalog**, and **Orders**.

Refactoring to Micro-Surgery

2. (20 min.) UML:

- Open your VS Code and look at you folder structure. Draw a **Component Diagram** of your current project.
  - **Logic Check**: You must draw separate boxes for `UserControllor`, `ProductService`, `OrderService`, etc.
  - **The "Cut" Line**: You must draw a dotted red line where they would physically "cut" the server into two separate machines.

Refactoring to Micro-Surgery

3. (10 min.) Peer Evaluation:

- Students swap drawings.
- **Critical Thinking Question**:
  - Look at your friend's diagram—are the services too dependent on each other?
  - Look at your friend's "cut" line. If they move the `OrderService` to a new server, does it still have access to the `Product` database? If not, how does it get the price?

Refactoring to Micro-Surgery

4. (5 min.) GenAI Prompt Engineering:

- Use your AI that integrated in your VS Code.

Prompt:

"Act as a Senior SA. Check my monolithic Express app. Project. I want to apply Separation of Concerns. Please:

1. Extract the Database queries into a Repository pattern.

2. Move the business logic into a Service layer.

3. Keep the Express routes in a separate Controller.

List what has been changed and explain why this makes the app easier to scale horizontally later.

Refactoring to Micro-Surgery

4. (5 min.) GenAI Prompt Engineering:

Prompt:

"*I have a modular Node.js app with a Controller-Route-Service pattern. I want to move my* `OrderService` *into a separate microservice. Looking at this code [Paste OrderService.js], what dependencies does it have on the* `Product` *or* `User` *modules? List the data it needs from other modules to function independently.*"

Refactoring to Micro-Surgery

5. (20 min.) Integration:

- Simulated Microservice
- You don't actually move the code. Instead, you "mock" a microservice call.
- **Task**: Replace a direct function call (e.g., `UserServide.findById()`) with a simulated fetch call (e.g., `fetch('http://user-service/api/verify')`).

Refactoring to Micro-Surgery

6. (10 min.) Debugging & Testing:

- **The Test**: Students "shut down" their User Controller logic.
- **The Critical Thinking Question**: "If your Auth service is 'down' (commented out), can a guest still see products? If your code crashes, you haven't decoupled your services enough."

Refactoring to Micro-Surgery

7. (5 min.) Version Control:

- Terminal Command:
  - `git add`
  - `git commit -m "docs: map component boundaries for future microservice migration.”`
  - `git push.`
- VS Code:
- GitHub Desktop:

Key Takeaway

Scalability and **Separation of Concerns**:

- **The "Pizza Team" Concept:** Explain that Microservices allow small teams to own one part of the app.
- **Technology Agnostic:** In a microservice world, the "Product" team can use a NoSQL database while the "Order" team uses SQLite.
- **Resilience:** If the "Recommendation Service" is slow, the "Checkout" shouldn't be affected.

Key Takeaway

- Modularization is how you organize your **Code**;
- Microservices are how you organize your **Business**.
- Your project is modular enough to be split—the question is, where do you draw the line to reduce failure points?